Introduction AMD Vivado/Vitis

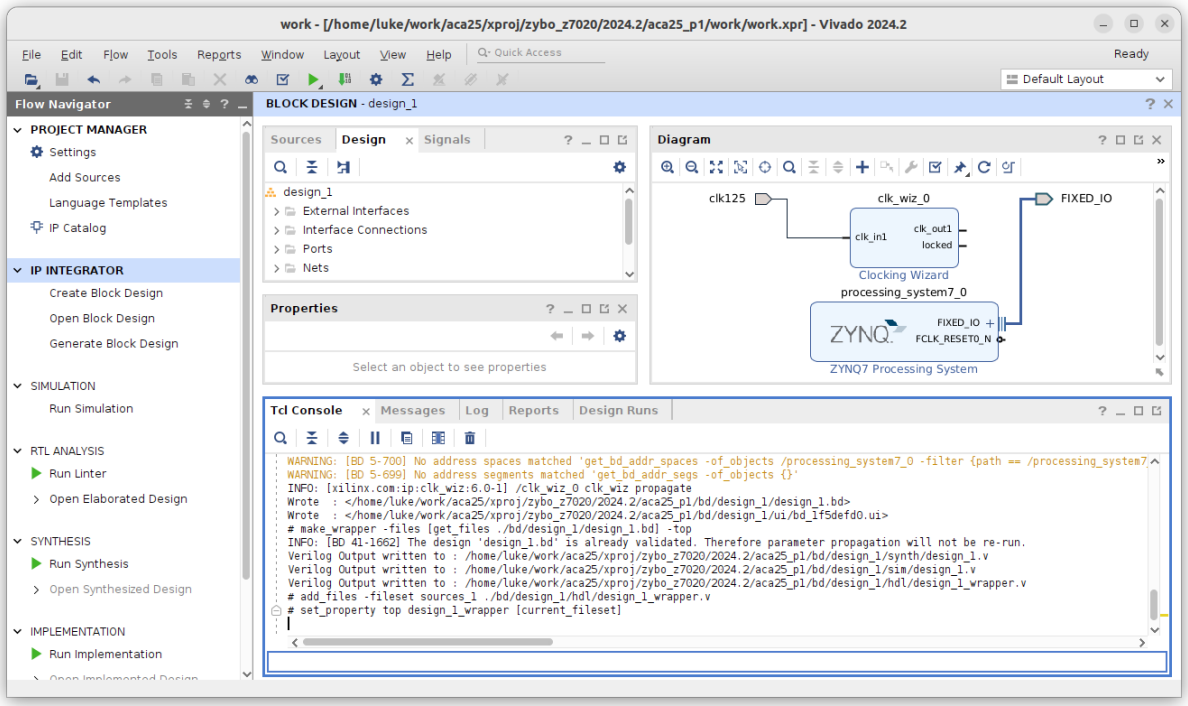
By the end of the lab you will be able to setup a microprocessor with peripherals in AMD vivado and write C software for it in AMD Vitis.

Getting Started

The first step is use Vivado to set up a hardware platform consisting of AMD's MicroBlaze processor, basic I/O peripherals, and supporting components. Obtain the lab code from moodle (or github: https://www.github.com/lukevassallo/hd-aca-25) and follow the next steps to set up a minimal Vivado project.

1. Unzip the code, or clone the repository and enter the directory.
2. Enter the container with the provided start script (**`bash start.sh`**) and launch vivado by typing the **`vivado`**. The vivado GUI will pop up. In the tcl console on the bottom left source the build_bd.tcl script with: **`source scripts/build_bd.tcl`**. This will create a new project with the expected result illustrated below.

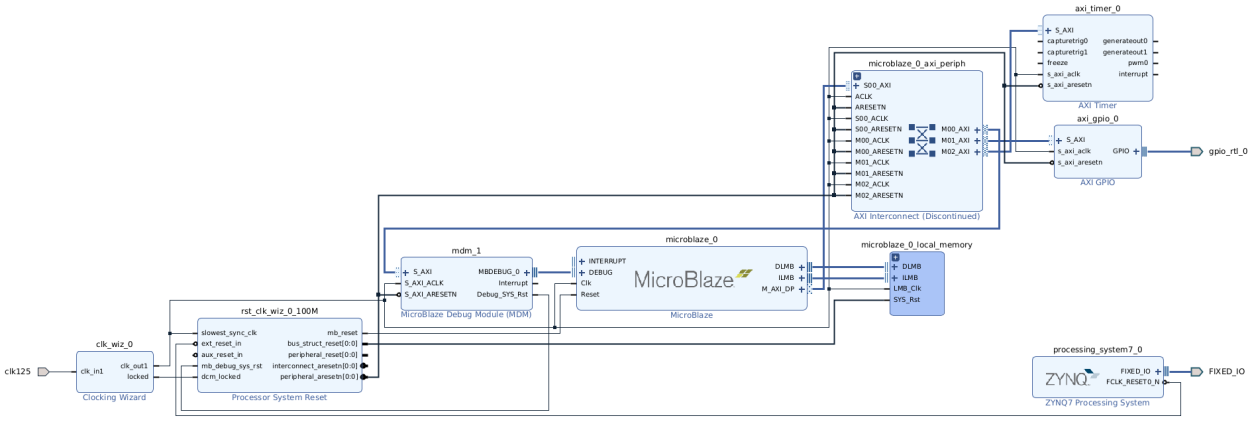
The build_bd.tcl script creates a basic project and sets up a vivado project targeting the Zybo Z7-20 development board. Afterwards it creates a block diagram and instantiates key IP blocks for clocking and reset.



Completing the System

If the block diagram is not already open, open it from the IP Integrator menu in the flow navigator (on the left) and instantiate the following components:

1. From the IP Integrator, add the MicroBlaze processor to the block diagram (context menu → Add IP → Microblaze). Run block automation and select the following configurations:
   - Set local memory to 128kB (plenty of space to store programs and data)
   - Debug module should be set to Debug & UART – The UART facilitates printing messages.
2. Add a general purpose Input/Output (GPIO) block and using *connection automation* or otherwise create an external port for it. The name of the port is expected to be the default: `gpio_rtl_0`. This port is connected the slide switches, push button switches, and LEDs on the development board (see Table 1 for mapping). You can explore the `.xdc` file and the Zybo Z7 schematic to learn more about this.
3. Add a timer following a similar procedure
4. Run automatic block/connection automation if needed, carefully reviewing the connections between the IP blocks.
5. Wire up any remaning connections, validate the design, and generate the bitstream



The 32-bit wide GPIO port is mapped to the board switches and leds for you. Table 1. summaries the mapping.

Table 1: Mapping between the design's GPIO port and Zybo Z7 switches and LEDs

Buttons / Switches		LEDs	
Design Port	**ZYBO Z7 Designator**	**Design Port**	**ZYBO Z7 Designator**
gpio_rtl_0[3:0]	BTN[3:0]	gpio_rtl_0[18:16]	LD5 – 16:RED, 17:GREEN, 18:BLUE
gpio_rtl_0[7:4]	SW[3:0]	gpio_rtl_0[21:19]	LD6 - 19:RED, 20:GREEN, 21:BLUE
gpio_rtl_0[15:8]	unused	gpio_rtl_0[25:22]	LD[3:0]

Writing Software

After generating the hardware platform in Vivado, use Vitis to develop firmware for the target processor. This step is performed in Vitis using the Xilinx Support Archive (.xsa), exported from Vivado. The .xsa file is sometimes also called the hardware handoff file, signalling the transition from hardware to software development.

The software environment requires knowledge of the underlying hardware in order to be able to compile code. As a result two projects need to be specified

- The platform project
- The application project.

The workflow is managed mostly through the application project, and the platform project runs in the background providing the tools with platform specific libraries.

The plaftorm project together with the .xsa file contains all information about the target hardware, it's capabilities and features.

1. After successful bitstream generation, export the hardware file from Vivado. You can close vivado after this step.
2. Launch vitis **`vitis --classic`** and specify the workspace location.
3. Create a platform project specifying the .xsa file in step 1.
4. Create an application project, selecting empty C application.
5. Copy the provided source files.

Lab Task

We will use the GPIO module to read input signals from push buttons BTN0, BTN1, and BTN2, and to control the output of the RGB LED, LD5. Using the documentation for the GPIO module, write C code to modify/control the configuration registers of the GPIO peripheral such that,

- pressing BTN0 turns on the red channel of LD5
- Pressing BTN1 turns on the green channel of LD5.
- Pressing BTN2 turns on the blue channel of LD5. If multiple buttons are pressed simultaneously multiple channels should light up.

Homework

Implement Pulse Width Modulation (PWM) in software using the timer and GPIO peripherals. The PWM signal should be used to control one channel (Red, Green, or Blue) of an RGB LED (LD4 or LD5). Use a button switch to adjust the duty cycle of the PWM in five discrete steps between 0% and 100%. Each press of the button should increment the duty cycle, cycling back to 0% after reaching 100%. One way to approach the task is:

1. Look up the documentation of the timer module, and understand how to configure it.
2. Using the timer establish a counting interval that meets the following specifications:
   - PWM resolution of 100 steps
   - PWM frequency of 10Hz
3. Using the GPIO monitor a button switch and upon detecting a press increase the duty cycle.

How does the LED react as you vary the PWM? The homework contributes 5% towards your final grade, and will be evaluated via demonstration.

Questions

When possible please post questions on the moodle forum (and tag us so that we get a notification).

References

1. Digilent Zybo Z7: https://digilent.com/reference/programmable-logic/zybo-z7/start
2. Digilent Zybo Z7 reference manual: https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual
3. Digilent Zybo Z7 schematic (Rev D.1): https://files.digilent.com/resources/programmable-logic/zybo-z7/zybo-z7-d1-sch.pdf
4. MicroBlaze Processor Reference Guide (UG984)

Appendix – A quick look at Microblaze organisation

Microblaze is a highly customisable Intellectual property (IP) core for a RISC microprocessor developed by Xilinx. It is designed for embedded applications, thus some architectural features are best understood through this perspective. Here is a succinct list of the key points:

- Single issue in-order pipelined processors with configurable number of stages (3/5/8)
- Optional integer and floating point unit
- Local memory bus with single cycle latency and optional AXI4 instruction and data caches
- Optional AXI4 (lite) memory-mapped IO Bus
- Optional memory management unit, capable of hosting operating system